

HELP DOCUMENTATION

API

How To

FAQ

Legend

TLE Format

Data Requests

Laser Clearinghouse

SGP4

User Agreement

Privacy

Contact Us

How to Use the API

Getting a Cookie

Download Data via cURL

Download Public Files

Using WGet

Using C#

Using C++

Using Java

Using Python

Getting the Delta

Disclaimer: There are many free software packages available for retrieving files using standard protocols. Space-Track.org does not endorse nor support any particular application, but we do provide these instructions and examples to help our users. Note that **we recommend using cURL (<http://curl.haxx.se/>)** because it can both send and retrieve data/files. Although we provide instructions for wget (<http://www.gnu.org/software/wget/>), our users have encountered more problems using wget than cURL.



How to login and get a cookie

space-track.org uses encrypted cookies for session handling. There are two ways to obtain the session cookie:

1. Graphical User Interface (GUI) users login normally using the web interface: <https://www.space-track.org/auth/login> (<https://www.space-track.org/auth/login>)
2. Scripted Application Programming Interface (API) users login by sending a HTTP POST request ('identity=your_username&password=your_password') to: <https://www.space-track.org/ajaxauth/login> (**examples below**)

WARNING: ALL scripts and programs should connect via <https://www.space-track.org/ajaxauth/login>. Using ANY other access URL will cause the attempt to fail.

How to extend the life of a cookie

Cookies are valid for 2 hours. To check if your current cookie is still valid and extend its lifetime by another 2 hours, make a GET request to <https://www.space-track.org/app/data/whoami> (<https://www.space-track.org/app/data/whoami>). This returns a JSON object with the following properties:

1. **logged_in**: A boolean true/false indicating whether you are authenticated or not. If false, you will need to log in again and get a new cookie to continue using the API.
2. **identity**: The user name of the account your cookie is authenticated as, or null if you are no longer logged in.
3. **session_expiration**: The expiration time of the current cookie, in ISO-8601 format. If you are logged in, this will always be 2 hours from the time you make your request.

How to logout and close a session

Logout by sending a <https://www.space-track.org/ajaxauth/logout> request (**examples below**)

↑ Back to top

How to return data through the API using cURL (recommended)



Step 1 (optional): If using a proxy server, set a local environment variable where "proxyhost" is the URL of your proxy and "portnumber" is the proxy's port number. (Thanks to Digital Globe for contributing this)

```
$ export https_proxy=http://proxyhost:portnumber
```

Step 2: obtain a session cookie:

- Cookies are good for ~2 hours after the last activity.
- Replace myusername and mY_S3cr3t_pA55w0rd! with your space-track.org username & password.
- Depending on the special characters in your password.
 - You may need to enclose the password in single or double quotes depending on your OS.
 - If that doesn't work, try Uuencoding (https://www.w3schools.com/tags/ref_urlencode.asp) the special character(s).
- If you are using a proxy from Step 1, append the command below with `--proxy-anyauth -U`.

```
$ curl -c cookies.txt -b cookies.txt https://www.space-track.org/ajaxauth/login -d "ic
```

Step 3: use the cookie to connect to <https://www.space-track.org/> and return the data in a RESTful query like these examples (<https://www.space-track.org/documentation/#api-sampleQueries>). You can construct your own API query using URL-safe (http://en.wikipedia.org/wiki/Percent-encoding#Character_data) characters if you need and substitute it for the one below:

```
$ curl --cookie cookies.txt https://www.space-track.org/basicspacedata/query/class/box
```

Step 4 (optional): direct the API query results into a file. Using the curl command from Step 2, redirect the query's output into a file (boxscore.json is only an example):

```
$ curl --cookie cookies.txt https://www.space-track.org/basicspacedata/query/class/box
```

↑ Back to top

How to download Public Files through the API

Notice

Effective 28 July 2025, Space-Track.org (/) will no longer provide SpaceX ephemerides for download on its "Public Files" page (/#publicFiles) or via the site's "publicfiles" API endpoint. To continue accessing these files, please transition to downloading individual ephemeris files directly from SpaceX. Follow the instructions on their website here: <https://www.starlink.com/public-files/ephemerides/README.md> (<https://www.starlink.com/public-files/ephemerides/README.md>)



Authenticate using the ajaxauth login, following the instructions from https://www.space-track.org/documentation#howto-api_curl (https://www.space-track.org/documentation#howto-api_curl)

Recommended Workflow

Step 0: Authenticate with the API, storing your cookies.

Step 1: View the available directories by performing a GET request to <https://www.space-track.org/publicfiles/query/class/dirs> (<https://www.space-track.org/publicfiles/query/class/dirs>)

Step 2: Get the detailed information for each file available by performing a GET request to <https://www.space-track.org/publicfiles/query/class/loadpublicdata> (<https://www.space-track.org/publicfiles/query/class/loadpublicdata>)

Step 3: Use the details to iterate and filter by whatever criteria you want (SOURCE, TYPE, DATE, and SIZE), until you get the name (LINK) of the file to download.

Step 4: Perform a GET request to the LINK for the file to download, and write to the output file.

View what directories contain Public Files

To list all directories that have public files, perform a GET request to <https://www.space-track.org/publicfiles/query/class/dirs> (<https://www.space-track.org/publicfiles/query/class/dirs>). This will return a JSON array of directories that contain public files:

```
curl --cookie cookies.txt https://www.space-track.org/publicfiles/query/class/dirs
```

Return will be of the format:

```
["directory_1", "directory_2", ..., "directory_n"]
```

View what files are available for download

To list all of the files that are available for download, perform a GET request to <https://www.space-track.org/publicfiles/query/class/files> (<https://www.space-track.org/publicfiles/query/class/files>). This will return a JSON object of directories, and the files within them:

```
$ curl --cookie cookies.txt https://www.space-track.org/publicfiles/query/class/files
```

Return will be of the format:

```
{"directory_1": ["directory_1_file_1", "directory_1_file_2", ..., "directory_1_file_n"]}
```

View more information about Public Files



To view more information about Public Files, perform a GET request to <https://www.space-track.org/publicfiles/query/class/loadpublicdata> (<https://www.space-track.org/publicfiles/query/class/loadpublicdata>). This will return a JSON-encoded array of objects with the structure:

SOURCE - source of the data

TYPE - type of the data

DATE - date at which the file was generated

LINK - where to download the file

NAME - the name of the file

SIZE - size of that specific file

```
$ curl --cookie cookies.txt https://www.space-track.org/publicfiles/query/class/loadpublicdata
```

Return will be of the format:

```
[{"SOURCE":"source_1","TYPE":"Ephemeris","DATE":"DATETIME","LINK":"directory_1_file1.txt","NAME":"source_1_file1.txt","SIZE":1024}]
```

Download a Public File

To download a public file, first determine which file you want to download (through the above methods). Then, perform a GET request to the endpoint, providing the 'LINK' value as the name of the file. Remember to provide the NAME as the output file cURL writes to.

```
curl -L --cookie cookies.txt https://www.space-track.org/publicfiles/query/class/loadpublicdata?LINK=directory_1_file1.txt
```

[↑ Back to top](#)

How to return data through the API using wget in Linux and Windows (cannot use wget to upload)

Using Linux

Step 1: obtain a session cookie good for ~2 hours (replace myusername and mY_S3cr3t_pA55w0rd!) with your space-track.org username & password):

```
$ wget --post-data='identity=myusername&password=mY_S3cr3t_pA55w0rd!' --cookies=cookies.txt https://www.space-track.org/publicfiles/query/class/loadpublicdata
```

(check the log file created for "Connecting to https://www.space-track.org/ ... connected" and "HTTP request sent, awaiting response... 200 OK")

Step 2: use the cookie to connect to https://www.space-track.org/ and return the data in a RESTful query like these examples (https://www.space-track.org/documentation/#api-sampleQueries). You can construct your own API query using URL-safe (http://en.wikipedia.org/wiki/Percent-encoding#Character_data) characters if you need to and substitute it for the one below:

```
$ wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track.org
```

(automatically saves file named "boxscore" to present directory)

Step 3 (optional): direct the API query results into a file using -O

```
$ wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track.org
```

(saves "box.json" to present directory)

Using Windows

NOTE: If you encounter certificate verification issues using wget on Windows, try adding the '--no-check-certificate' option to both Step 2 and Step 3.

Step 1: download wget for windows (http://gnuwin32.sourceforge.net/packages/wget.htm) if you don't already have it installed. We recommend installing the 'setup' version because it includes all dependencies.

Step 2: obtain a session cookie good for ~2 hours (replace myusername and mY_S3cr3t_pA55w0rd! with your space-track.org username & password):

```
C:\> wget --post-data "identity=myusername&password=mY_S3cr3t_pA55w0rd!" --keep-s
```

(check the log file created for "Connecting to https://www.space-track.org/ ... connected" and "HTTP request sent, awaiting response... 200 OK")

Step 3: use the cookie to connect to https://www.space-track.org/ and return the data in a RESTful query like these examples (https://www.space-track.org/documentation/#api-sampleQueries). You can construct your own API query using URL-safe (http://en.wikipedia.org/wiki/Percent-encoding#Character_data) characters if you need to and substitute it for the one below:

```
C:\> wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track
```

(automatically saves folder named "boxscore" to present directory)

Step 4 (optional): direct the API query results into a file using -O



```
C:\> wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track.org
```

(saves "box.txt" to present directory)

Some Windows users have reported problems using the % character in scripts. If you encounter this issue, you may try escaping the character using two % instead of one;

e.g. /EPOCH/%%3Enow-30/ instead of /%3Enow-30/

```
C:\> wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track.org
```

(saves "amateur.txt" to present directory)

Optional Method: If you no longer need the cookie, use the log out function to close the session

```
$ wget --keep-session-cookies --load-cookies=cookies.txt https://www.space-track.org
```

↑ Back to top

How to connect through the API using C#

NOTE: The following code was sent in by Mr. R. Bovard. We offer it here as-is, with no warranty implied.



```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net;
using System.IO;
using System.Collections.Specialized;

namespace GetTLEData
{
    public class SpaceTrack
    {
        public class WebClientEx : WebClient
        {
            // Create the container to hold all Cookie objects
            private CookieContainer _cookieContainer = new CookieContainer();

            // Override the WebRequest method so we can store the cookie
            // container as an attribute of the Web Request object
            protected override WebRequest GetWebRequest(Uri address)
            {
                WebRequest request = base.GetWebRequest(address);

                if (request is HttpWebRequest)
                    (request as HttpWebRequest).CookieContainer = _cookieContainer;

                return request;
            }
        } // END WebClient Class

        // Get the TLEs based of an array of NORAD CAT IDs, start date, and end date
        public string GetSpaceTrack(string[] norad, DateTime dtstart, DateTime dtend)
        {
            string uriBase = "https://www.space-track.org";
            string requestController = "/basicspacedata";
            string requestAction = "/query";
            // URL to retrieve all the latest tle's for the provided NORAD CAT
            // IDs for the provided Dates
            //string predicateValues = "/class/gp/NORAD_CAT_ID/" + string.Join(",",
            // URL to retrieve all the latest 3le's for the provided NORAD CAT
            // IDs for the provided Dates
            string predicateValues = "/class/gp_history/EPOCH/" + dtstart.ToString("
            string request = uriBase + requestController + requestAction + p

            // Create new WebClient object to communicate with the service
            using (var client = new WebClientEx())
            {
                // Store the user authentication information
                var data = new NameValueCollection
                {
                    { "identity", "myUserName" },
                    { "password", "myPassword" },

```




```
};

// Generate the URL for the API Query and return the response
var response2 = client.UploadValues(uriBase + "/ajaxauth/login", data)
var response4 = client.DownloadData(request);

return (System.Text.Encoding.Default.GetString(response4));
}
} // END GetSpaceTrack()
} // END SpaceTrack Class
} // END namespace GetTLEData
```

↑ Back to top

How to connect through the API using C++

NOTE: The following code was sent in by Mr. A. Wenz of BINARY SPACE. This is a C++ solution using the Microsoft® Foundation Classes (MFC) and compiles with Microsoft® Visual Studio® 2010 or higher. We offer it here as-is, with no warranty implied.

Header File:



```
#include <afxwin.h>      // MFC core and standard components
#include <afxext.h>       // MFC extensions

#include <afxinet.h>

// CSpaceTrackDownload
class CSpaceTrackDownload : public COBJECT
{
// Construction
public:
    CSpaceTrackDownload();
    CSpaceTrackDownload(LPCTSTR pszUrl, LPCTSTR pszUserName, LPCTSTR pszPassword);
    ~CSpaceTrackDownload();

// Attributes
private:
    DWORD    m_dwFlags;
    CHttpConnection *m_pHttpConnection;
    CInternetSession m_cInternetSession;

// Operations
public:
    BOOL Authenticate(LPCTSTR pszUrl, LPCTSTR pszUserName, LPCTSTR pszPassword);

    CString Query(LPCTSTR pszQuery);
};
```

Source Code:

```

#include "SpaceTrack.h"

// CSpaceTrackDownload
CSpaceTrackDownload::CSpaceTrackDownload() : CObject()
{
    m_dwFlags = 0;
    m_pHttpConnection = (CHttpConnection *) NULL;
}

CSpaceTrackDownload::CSpaceTrackDownload(LPCTSTR pszUrl, LPCTSTR pszUserName, LPCTSTR pszPassword)
{
    Authenticate(pszUrl, pszUserName, pszPassword);
}

CSpaceTrackDownload::~CSpaceTrackDownload()
{
    if (m_pHttpConnection != (CHttpConnection *) NULL)
    {
        m_pHttpConnection -> Close();
        delete m_pHttpConnection;
    }
}

BOOL CSpaceTrackDownload::Authenticate(LPCTSTR pszUrl, LPCTSTR pszUserName, LPCTSTR pszPassword)
{
    UINT    cbData;
    DWORD   dwType;
    LPSTR    pszData;
    CString  szPath;
    CString  szReply;
    CString  szServer;
    CString  szCredentials;
    CHttpFile *pHttpFile;
    INTERNET_PORT nPort;
    BYTE    nData[1024];

    for (pHttpFile = (CHttpFile *) NULL, pszData = (LPSTR) NULL; AfxParseURL(pszUrl, dwType, nPort, pszData);)
    {
        try
        {
            if ((m_pHttpConnection = m_cInternetSession.GetHttpConnection(szServer, (m_dwFlags & HTTP_FLAG_SECURE ? INTERNET_FLAG_SECURE : 0), pszData, dwType, nPort, 0)) != NULL)
            { if ((pHttpFile = m_pHttpConnection -> OpenRequest(CHttpConnection::HTTP_VERB_GET, szPath, szCredentials.Format(STRING(IDS_SPACETRACK_CREDENTIALSFORMAT) /* identifier for the credentials format */)) != NULL)
            {
                #ifndef UNICODE
                    strcpy_s(pszData, szCredentials.GetLength()+1, szCredentials);
                #else
                    WideCharToMultiByte(CP_ACP, 0, szCredentials, -1, pszData, szCredentials.GetLength(), 0, 0);
                #endif

                for (pHttpFile -> SendRequest(STRING(IDS_SPACETRACK_HTTPCONTENTTYPEHEADER), "text/plain");)
                {
                    #ifndef UNICODE
                        szReply = (LPCSTR) nData;
                    #endif
                }
            }
        }
        catch (...)
        {
            if (m_pHttpConnection != NULL) m_pHttpConnection -> Close();
            delete m_pHttpConnection;
            m_pHttpConnection = NULL;
        }
    }
}

```



```

#else
    MultiByteToWideChar(CP_ACP, 0, (LPCSTR) nData, -1, szReply.GetBufferSetLength,
    szReply.ReleaseBuffer());
#endif

    if (szReply == STRING(IDS_SPACETRACK_LOGINSUCCESS)) /* "" i.e. returns t
    { pHttpRequest -> Close();
      GlobalFree(pszData);
      delete pHttpRequest;
      return TRUE;
    }
    break;
  }
  GlobalFree(pszData);
  break;
}
pHttpRequest -> Close();
delete pHttpRequest;
}
m_pHttpRequest -> Close();
delete m_pHttpRequest;
}
break;
}
catch (...)
{
    if (pszData != (LPSTR) NULL) GlobalFree(pszData);
    if (pHttpRequest != (CHttpFile *) NULL)
    { pHttpRequest -> Close();
      delete pHttpRequest;
    }
    if (m_pHttpRequest != (CHttpConnection *) NULL)
    { m_pHttpRequest -> Close();
      delete m_pHttpRequest;
    }
    break;
}
}
m_dwFlags = 0;
m_pHttpRequest = (CHttpConnection *) NULL;
return FALSE;
} // END Authenticate()

CString CSpaceTrackDownload::Query(LPCTSTR pszQuery)
{
    UINT cbData;
    CString szResult[2];
    CHttpRequest *pHttpRequest;
    BYTE nData[65536];

    for (pHttpRequest = (CHttpFile *) NULL; m_pHttpRequest != (CHttpConnection *) NULL;
    {
        try

```



```

{
    if ((pHttpFile = m_pHttpConnection -> OpenRequest(CHttpConnection::HTTP_VERB_GET,
        { for (pHttpFile -> SendRequest(), ZeroMemory(nData, sizeof(nData)/sizeof(BYTE));
        {
#ifdef UNICODE
            szResult[0] += (LPCSTR) nData;
#else
            MultiByteToWideChar(CP_ACP, 0, (LPCSTR) nData, cbData, szResult[1].GetBufferSetLength(),
            szResult[1].ReleaseBuffer();
            szResult[0] += szResult[1];
#endif
        }
        pHttpFile -> Close();
        delete pHttpFile;
    }
    break;
}
catch (...)
{
    if (pHttpFile != (CHttpFile *) NULL)
    { pHttpFile -> Close();
      delete pHttpFile;
    }
    szResult[0].Empty();
    break;
}
}
return szResult[0];
} // END Query()

```

Example Use:

```

CString  szResult[2];
CSpaceTrackDownload  cDownload(TEXT("https://www.space-track.org/ajaxauth/login"),TEXT(""));

szResult[0] = cDownload.Query(TEXT("/basicspacedata/query/class/gp/orderby/TLE_LINE0%20ASC"));
szResult[1] = cDownload.Query(TEXT("/basicspacedata/query/class/gp/orderby/TLE_LINE0%20ASC"));

```

↑ Back to top

How to connect through the API using Java

NOTE: The following code was sent in by one of our users. We offer it here as-is, with no warranty



implied.



```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.CookieHandler;
import java.net.CookieManager;
import java.net.CookiePolicy;
import java.net.URL;
import javax.net.ssl.HttpURLConnection;

public class DownloadTLE {

    /**
     * @param args
     */
    public static void main(String[] args) {
        try {
            String baseUrl = "https://www.space-track.org";
            String authPath = "/ajaxauth/login";
            String userName = "USERNAME";
            String password = "PASSWORD";
            String query = "/basicspacedata/query/class/gp/EPOCH/%3Enow-10/format/3le'

            CookieManager manager = new CookieManager();
            manager.setCookiePolicy(CookiePolicy.ACCEPT_ALL);
            CookieHandler.setDefault(manager);

            URL url = new URL(baseUrl+authPath);

            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setDoOutput(true);
            conn.setRequestMethod("POST");

            String input = "identity="+userName+"&password="+password;

            OutputStream os = conn.getOutputStream();
            os.write(input.getBytes());
            os.flush();

            BufferedReader br = new BufferedReader(new InputStreamReader((conn.getInput

            String output;
            System.out.println("Output from Server .... \n");
            while ((output = br.readLine()) != null) {
                System.out.println(output);
            }

            url = new URL(baseUrl + query);

            br = new BufferedReader(new InputStreamReader((url.openStream())));

            while ((output = br.readLine()) != null) {
```



```
        System.out.println(output);
    }

    url = new URL(baseUrl + "/ajaxauth/logout");
    br = new BufferedReader(new InputStreamReader((url.openStream())));
    conn.disconnect();

} catch (Exception e) {
    e.printStackTrace();
}

}
```

[↑ Back to top](#)

How to connect through the API using Python

NOTE: The following code was provided by Mr. A. Stokes. We offer it here as-is, with no warranty implied. Before running this script, certain dependencies must be installed and created. Please ensure a separate config file named SLTrack.ini is created in the same directory as the python script. Also, Python 3.x and the PIP Python package manager must be installed prior to using PIP to install configparser and xlswriter.

NOTE: Please be aware that this example specifically requests historical SpaceX Starlink data.

SLTrack.ini file (where XXX and YYY are your space-track.org credentials and ZZZ is your Excel Output file name - e.g. starlink.xlsx):

```
[configuration]
username = XXX
password = YYY
output = ZZZ
```

Source Code:




```

##
## SLTrack.py
## Derived from original (c) 2019 Andrew Stokes
##
## Simple Python app to extract Starlink satellite history data from www.space-track.org
## (Note action for you in the code below, to set up a config file with your access an
##
##
## Copyright Notice:
##
## This program is free software: you can redistribute it and/or modify
## it under the terms of the GNU General Public License as published by
## the Free Software Foundation, either version 3 of the License, or
## (at your option) any later version.
##
## This program is distributed in the hope that it will be useful,
## but WITHOUT ANY WARRANTY; without even the implied warranty of
## MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
## GNU General Public License for more details.
##
## For full licencing terms, please refer to the GNU General Public License
## (gpl-3_0.txt) distributed with this release, or see
## http://www.gnu.org/licenses/.
##

import requests
import json
import configparser
import xlswriter
import time
from datetime import datetime

class MyError(Exception):
    def __init__(self, args):
        Exception.__init__(self, "my exception was raised with arguments {}".format(ar
        self.args = args

# See https://www.space-track.org/documentation for details on REST queries
# the "Find Starlinks" query searches all satellites with NORAD_CAT_ID > 40000, with C
# the "OMM Starlink" query gets all Orbital Mean-Elements Messages (OMM) for a specifi

uriBase = "https://www.space-track.org"
requestLogin = "/ajaxauth/login"
requestCmdAction = "/basicspacedata/query"
requestFindStarlinks = "/class/gp/NORAD_CAT_ID/>40000/OBJECT_NAME/STARLINK~~/format/"
requestOMMStarlink1 = "/class/gp/NORAD_CAT_ID/"
requestOMMStarlink2 = "/orderby/EPOCH/format/xml"

# Parameters to derive apoapsis and periapsis from mean motion (see https://en.wikiped

GM = 398600441800000.0
GM13 = GM ** (1.0/3.0)

```



```

MRAD = 6378.137
PI = 3.14159265358979
TPI86 = 2.0 * PI / 86400.0

# ACTION REQUIRED FOR YOU:
#=====
# Provide a config file in the same directory as this file, called SLTrack.ini, with t
# [configuration]
# username = XXX
# password = YYY
# output = ZZZ
#
# ... where XXX and YYY are your www.space-track.org credentials (https://www.space-tr
# ... and ZZZ is your Excel Output file - e.g. starlink-track.xlsx (note: make it an .

# Use configparser package to pull in the ini file (pip install configparser)
config = configparser.ConfigParser()
config.read("./SLTrack.ini")
configUsr = config.get("configuration","username")
configPwd = config.get("configuration","password")
configOut = config.get("configuration","output")
siteCred = {'identity': configUsr, 'password': configPwd}

# User xlswriter package to write the xlsx file (pip install xlswriter)
workbook = xlswriter.Workbook(configOut)
worksheet = workbook.add_worksheet()
z0_format = workbook.add_format({'num_format': '#,##0'})
z1_format = workbook.add_format({'num_format': '#,##0.0'})
z2_format = workbook.add_format({'num_format': '#,##0.00'})
z3_format = workbook.add_format({'num_format': '#,##0.000'})

# write the headers on the spreadsheet
now = datetime.now()
nowStr = now.strftime("%m/%d/%Y %H:%M:%S")
worksheet.write('A1', 'Starlink data from' + uriBase + " on " + nowStr)
worksheet.write('A3', 'NORAD_CAT_ID')
worksheet.write('B3', 'SATNAME')
worksheet.write('C3', 'EPOCH')
worksheet.write('D3', 'Orb')
worksheet.write('E3', 'Inc')
worksheet.write('F3', 'Ecc')
worksheet.write('G3', 'MnM')
worksheet.write('H3', 'ApA')
worksheet.write('I3', 'PeA')
worksheet.write('J3', 'AvA')
worksheet.write('K3', 'LAN')
worksheet.write('L3', 'AgP')
worksheet.write('M3', 'MnA')
worksheet.write('N3', 'SMa')
worksheet.write('O3', 'T')
worksheet.write('P3', 'Vel')
wsline = 3

```



```

# use requests package to drive the RESTful session with space-track.org
with requests.Session() as session:
    # run the session in a with block to force session to close if we exit

    # need to log in first. note that we get a 200 to say the web site got the data, r
    resp = session.post(uriBase + requestLogin, data = siteCred)
    if resp.status_code != 200:
        raise MyError(resp, "POST fail on login")

    # this query picks up all Starlink satellites from the catalog. Note - a 401 failu
    resp = session.get(uriBase + requestCmdAction + requestFindStarlinks)
    if resp.status_code != 200:
        print(resp)
        raise MyError(resp, "GET fail on request for Starlink satellites")

    # use the json package to break the json formatted response text into a Python str
    retData = json.loads(resp.text)
    satCount = len(retData)
    satIds = []
    for e in retData:
        # each e describes the latest elements for one Starlink satellite. We just need
        catId = e['NORAD_CAT_ID']
        satIds.append(catId)

    # using our new list of Starlink satellite NORAD_CAT_IDs, we can now get the OMM n
    maxs = 1
    for s in satIds:
        resp = session.get(uriBase + requestCmdAction + requestOMMStarlink1 + s + requ
        if resp.status_code != 200:
            # If you are getting error 500's here, its probably the rate throttle on t
            # wait a while and retry
            print(resp)
            raise MyError(resp, "GET fail on request for Starlink satellite " + s)

    # the data here can be quite large, as it's all the elements for every entry f
    retData = json.loads(resp.text)
    for e in retData:
        # each element is one reading of the orbital elements for one Starlink
        print("Scanning satellite " + e['OBJECT_NAME'] + " at epoch " + e['EPOCH'])
        mmoti = float(e['MEAN_MOTION'])
        ecc = float(e['ECCENTRICITY'])
        worksheet.write(wsline, 0, int(e['NORAD_CAT_ID']))
        worksheet.write(wsline, 1, e['OBJECT_NAME'])
        worksheet.write(wsline, 2, e['EPOCH'])
        worksheet.write(wsline, 3, float(e['REV_AT_EPOCH']))
        worksheet.write(wsline, 4, float(e['INCLINATION']),z1_format)
        worksheet.write(wsline, 5, ecc,z3_format)
        worksheet.write(wsline, 6, mmoti,z1_format)
        # do some ninja-fu to flip Mean Motion into Apoapsis and Periapsis, and to
        sma = GM13 / ((TPI86 * mmoti) ** (2.0 / 3.0)) / 1000.0
        apo = sma * (1.0 + ecc) - MRAD

```



```

per = sma * (1.0 - ecc) - MRAD
smak = sma * 1000.0
orbT = 2.0 * PI * ((smak ** 3.0) / GM) ** (0.5)
orbV = (GM / smak) ** (0.5)
worksheet.write(wsline, 7, apo,z1_format)
worksheet.write(wsline, 8, per,z1_format)
worksheet.write(wsline, 9, (apo+per)/2.0,z1_format)
worksheet.write(wsline, 10, float(e['RA_OF_ASC_NODE']),z1_format)
worksheet.write(wsline, 11, float(e['ARG_OF_PERICENTER']),z1_format)
worksheet.write(wsline, 12, float(e['MEAN_ANOMALY']),z1_format)
worksheet.write(wsline, 13, sma,z1_format)
worksheet.write(wsline, 14, orbT,z0_format)
worksheet.write(wsline, 15, orbV,z0_format)
wsline = wsline + 1
maxs = maxs + 1
if maxs > 18:
    print("Snoozing for 60 secs for rate limit reasons (max 20/min and 200/hr)
    time.sleep(60)
    maxs = 1
session.close()
workbook.close()
print("Completed session")

```

↑ Back to top

How to download just the changes (i.e. the data added to space-track) since your last download

For TLEs, you can use the `tle_publish` (https://www.space-track.org/basicspacedata/modeldef/class/tle_publish/format/html) class used on the Recent TLE (<https://www.space-track.org/#recent>) page to tell what TLEs were published since your last download. This class shows the date/time the TLE was published to Space-Track.org, but only in two-line format.

The procedure below uses the FILE predicate and applies identically to all of these API classes: `tle`, `satcat`, `decay`, and `cdm`. If you have an alternate method to return only the changes, please email us (<mailto:admin@space-track.org>).

Step 1: establish a baseline by returning the latest TLE for objects that you want to monitor. In the example below, we use four objects 25544,36411,26871, and 27422. You can use any criteria to determine the objects you want to monitor. You can also return any predicates you need in any format. We show FILE, EPOCH, and the two lines of the TLE as an example:

```
https://www.space-track.org/basicspacedata/query/class/gp/NORAD_CAT_ID/25544,36411,26871,27422,FILE,EPOCH,2
```

Results:

FILE	EPOCH	TLE_LINE1
	TLE_LINE2	
1339414	2012-08-23 14:41:35	1 25544U 98067A 12236.61221542 .00014781 00000-0 26483
1660	2 25544	051.6476 145.7539 0014860 352.2902 087.3290 15.50074566788503
1339388	2012-08-23 13:09:41	1 36411U 10008A 12236.54839284 .00000068 00000-0 00000
1996	2 36411	000.4393 261.8677 0004087 269.6508 223.6226 01.00284253 9094
1339308	2012-08-23 09:04:34	1 26871U 01031A 12236.37818175 -.00000300 00000-0 10000
7001	2 26871	002.4043 072.3013 0002099 076.5029 259.0419 01.00285194 40660
1339234	2012-08-23 05:26:49	1 27422U 02021B 12236.22696129 .00000039 00000-0 32070
9902	2 27422	098.3978 295.8841 0013298 043.4759 316.7468 14.28769327535462

Step 2: determine the highest file number for the set of objects you want to monitor. File numbers are sequential and incremented each time space-track receives a batch of TLEs or a single TLE. They are not unique identifiers of individual TLEs, but they can serve as a "bookmark" so you know where to begin pulling data the next time you visit.

```
https://www.space-track.org/basicspacedata/query/class/gp/NORAD_CAT_ID/25544,36411,268
```

In our case, for the four objects we care about, this query returned one column with one row: 1339414 (verified by visually inspecting the results in step 1). **STORE THIS VALUE in your system as a "bookmark" for use during your next data pull.**

Step 3: When you pull data from space-track again on the following day, query for every record with a file number higher than our bookmark from step 2 (1339414 in our example). **We request that you query space-track not more than once every 4 hours.** The data provider does not deliver updates often enough to justify a higher frequency.

```
https://www.space-track.org/basicspacedata/query/class/gp_history/format/html/NORAD_CAT_ID/1339414
```

Results:

FILE	TLE_LINE1	TLE_LINE2
1339478	1 25544U 98067A 12236.74387093 .00004357 00000-0 83943-4 0 1672 2 2554	

Step 4: You need to store the new highest file number (1339478) as the "bookmark" for your next data pull on the following day (or week). You can either iterate through the data on your end to determine the highest file number (recommended) or run this API query immediately before the one in step 3 and store the value as your bookmark for the next pull:



<https://www.space-track.org/basicspacedata/query/class/gp/predicates/FILE/orderby/FILE>

